

# Locating On-Street Parking through Semantic Segmentation on Street View Images

Mufaro Machaya<sup>1</sup>\*, Rodrigo Moreno González<sup>1†</sup>, and Bavisha Kalyan<sup>1‡</sup>

August 2026

<sup>1</sup>University of British Columbia

## Abstract

*As population density grows, cities are looking for smart, technology-driven methods for managing public land, including on-street parking. Precise, up-to-date information on on-street parking locations could aid the development of efficient real-time parking systems and mixed-use development efforts. As such, in partnership with Neighborhood, a community organization in Oakland interested in leveraging on-street parking land for developing parklets for housing and community enrichment, this study seeks to develop a method for automatically locating and cataloguing on-street parking through the semantic segmentation of Street View Images (SVIs). To do so, we developed a data analysis pipeline for obtaining and labeling 800 SVIs of a neighborhood in Oakland from Google Street View, fine-tuned SegFormer B-2 on these images to 72% classification accuracy and 52% IoU (intersection over union), then applied this model at a uniform distance across all roads in the neighborhood to determine its parking spots. These boundaries were then transformed out of the images they were classified on to a GPS map using a homographic transform, and a final GPS map is produced from these transformations after a series of post-processing techniques are applied to circumvent random errors in the dataset and model. Through this, we show that although this process is non-negligibly error-prone due to having multiple steps that each introduce unique, compounding inaccuracies, it is overall highly cost-efficient and ideal for urban development.*

## 1 Introduction

Due to recent technological advances, growing populations, and aging present city management systems, many urban centers within the past decade have been seeking to transition into “smart cities” through technology-integrated services [1, 2]. One such clear case is in the present problem of on-street parking, as the lack of many standard, scalable systems for tracking and managing on-street parking inventory in an age of self-driving cars and automatic routing systems (such as Google Maps) has led many to seek new methods for real-time free parking detection [3, 4, 5, 6]. Throughout these cases, however, the predominant perspective on the on-street parking space problem is particularly on real-time parking for drivers, but with growing adoption of tiny urban structures (TUS), such as food trucks, community gardens, or tiny homes [7, 8], it may be time for a fresh perspective on on-street parking space as publicly-owned land viable for development, much more than simply transient space for drivers.

The non-profit organization Neighborhood, located in Oakland, CA, USA, holds this vision, and, thus, promotes the integration of tiny homes across Oakland [9]. For Neighborhood, the recent Oakland Parklet Program, designed to convert

on-street parking into space for development [7], can serve as a test case for the integration of TUS across Oakland. The immediate problem hindering this development, however, is that most cities lack up-to-date, complete maps of their on-street parking land inventory, if at all [10], as without knowledge of where TUS can be placed across a city, a wide-scale integration is not possible. Our aim in this paper, then, is to seek for a method for detecting, delineating and mapping on-street parking land, which would allow Neighborhood to better advocate for the integration of TUS in Oakland.

The solution we present is to apply image segmentation via a deep learning model on street view images (SVIs). Image segmentation on SVIs is an established, common method for building geospatial datasets in urban environments. The ease of data collection and the high image quality for SVIs make it preferable to other collection methods, such as aerial or satellite imagery [11]. Our model queries for SVI data along the roads of a city, segments the SVIs for on-parking spaces, transforms these segmented spaces into geographical coordinates and projects them into a city map. This gives us a comprehensive map of the on-street parking spaces across a city. We posit that this data analysis pipeline could be simultaneously scalable, efficient, and cost-effective enough to meet demands for information on on-street parking land capacity in urban environments.

---

\*mufaro2@student.ubc.ca

†rmorenog@student.ubc.ca

‡Corresponding author: bavisha.kalyan@ubc.ca

## 2 Literature Review

The majority of the literature concerning SVI segmentation applied to the recognition of on-street parking area consists of the detection and/or segmentation of driver or traffic camera imagery largely due to the ease of data collection using public traffic cameras, roadside cameras, and cameras mounted directly on drivers' vehicles [3]. For example, Gkolias and Vlahogianni leveraged SVI segmentation to detect empty on-street parking spaces with high (90%) accuracy [12]. Similarly, Gören et al. developed a traffic camera segmentation system that demonstrated the efficiency of deep learning applied to on-street parking detection [13]. Both papers demonstrate clear examples of the efficiency and viability of deep learning for on-street parking detection. Alahmadi et al. reviewed on-street parking detection methods from the perspective of scalability, efficiency, and privacy, and they demonstrated that "crowdsensing" vehicle-mounted camera detection has a strong potential to be low-cost and scalable at the risk of the privacy of drivers [5].

Furthermore, recent literature has demonstrated the efficacy of SVI analysis for urban mapping problems. Biljecki and Ito reviewed SVI analysis for urban analytics and mapping problems, demonstrating that SVI analysis is a method rapidly growing in popularity due to the ease of data collection, the volume of data immediately available, and the high visual and temporal granularity of SVI databases [11]. Therefore, we can conclude from previous research that SVI segmentation will likely be satisfactorily computationally efficient, scalable, and low-cost to meet the demands of city mapping, as an SVI mapping infrastructure would be highly dynamic and require no permanent infrastructure investment.

Furthermore, prior researchers demonstrated that SVI image analysis can be used to extract detailed information for building wide-scale geographic maps. For example, Pu et al. mapped out ideal locations for developing electric vehicle charging stations by segmenting roadside SVIs for free space, then performing geometric analysis to estimate the area of the space, allowing them to map out the density of potential electric vehicle development land across Singapore [14], and Sijabat et al. used SVIs and deep learning techniques in Bandung, Indonesia to map and classify city areas between slum and non-slum areas [15]. These papers demonstrate how mass SVI datasets over large geographic regions can be analyzed with machine learning to construct greater geographic maps, and they inspired us to produce a final map of on-street parking spaces through segmenting SVIs then performing geographic transformations on the produced segmented polygons.

On the other hand, within parking lots, there have been multiple successful attempts to extract information from parking images via machine learning models. For example Rahmat

et al. used security camera pictures to detect open parking spaces, with extremely high accuracy [16]. For our purposes, however, we would like to highlight the work of Koskovich, Starek and King on "VirtuaLot", wherein they used security cameras to extract information describing a parking lot to then be plotted onto an aerial image taken using an uncrewed aircraft system [17], and this study serves as a key example of how geographic information obtained from non-aerial imagery can be transformed into a 2D representation of the space [17].

As for prior attempts to build greater on-street parking maps, there have been multiple attempts with a variety of methods. For example, Navarro-B et al. obtained raw data of parking events (consisting of location, duration, and time and date) from the now-defunct German car-sharing service SHARE NOW and developed two methods for classifying whether each event occurred on a valid on-street parking space: spatial aggregation and machine learning [18]. However, this method lacks the ability to produce precise geographic boundaries for on-street parking land, and it is fundamentally limited by the assumption that users of car-sharing services will park in a way that is representative of the full on-street parking inventory for a city. Furthermore, as discussed by Alahmadi et al., inherently crowdsourced methods of data collection have the downside of being privacy-invasive to users and potentially over-relying on user input for data collection [5].

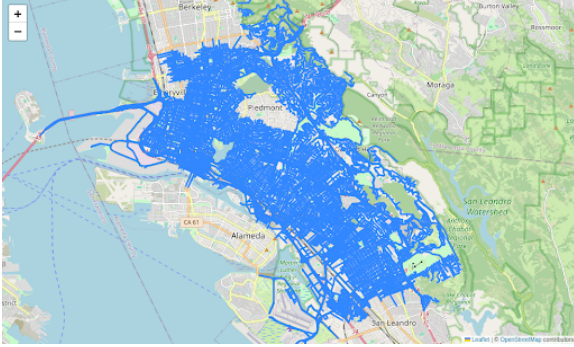
A different method for estimating parking space was tried in Henry et al. 2021 and further improved upon by Hellekes et al. in 2023 [19, 20]. By using satellite imagery and segmentation models, they aim to find the total parking area, including on-street parking, within a city. For on-street specific parking, they combine satellite imagery, segmented with ML, and land-use data to obtain an on-street specific figure. However, this process requires multiple ML models and a huge amount of aerial images to successfully classify on-street parking, as it relies on mutual comparison via a naive Bayesian fusion layer. Furthermore, this map can find parkable space, but not where the actual parking spots are (e.g. how many parking spots are in the parking area), which we needed for our own mapping attempt.

Altogether, prior literature has proven SVI analysis to both efficient and precise for building geographic maps [3], and in particular, utilizing projective transforms for analyzing images can be an effective strategy for extracting information on physical traits (such as the area or boundaries) of objects or regions captured within the image [14].

### 3 Methods

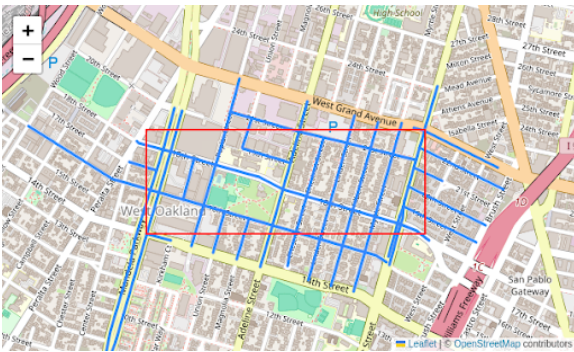
#### 3.1 Data Collection

To begin, we performed an Overpass Turbo query (see appendix A.1) to obtain a geographic map of the road geometry across Oakland in GeoJSON format, which can be seen in figure 1 [21]. This data was then analyzed using GeoPandas version 1.1.4 [22].



**Figure 1:** The result of the raw Overpass Turbo query for all of the road geometry across Oakland.

From there, we added a filter to only include the road geometry within our testing area between the intersections of 14th Street/Mandela Parkway and West Grand Avenue/Market Street, as seen in figure 2. This neighborhood was specifically picked as it is primarily residential with highly uniform and straight roadways, which would ensure a relatively high proportion of on-street parking space and simplify the detection process by reducing errors due to variations in the road.



**Figure 2:** The filtered road geometry map for the neighborhood between 14th Street/Mandela Parkway and West Grand Avenue/Market Street in Oakland, California.

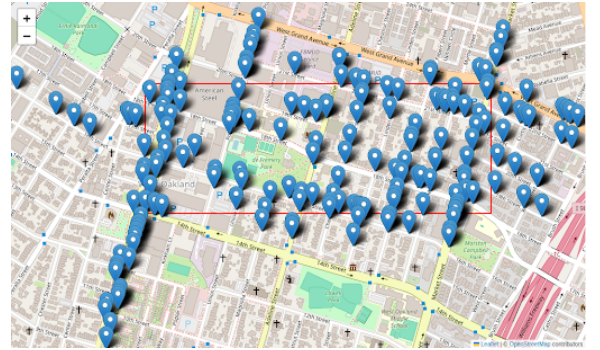
Then, Google’s Street View API was utilized to sample SVIs for a given GPS position and heading direction. In particular, given that each road is straight, for every GPS position along a roadway, the four heading directions parallel

and perpendicular to the road (which we denote as “principal” directions) are ideal for obtaining SVIs. Therefore, for every GPS point  $\vec{p}$  used for data collection, we sample four images, one for each principal direction  $\theta_n$  (see appendix A.2). These directions are calculated by obtaining two more points  $\vec{p}_1$  and  $\vec{p}_2$  along the roadway that are each an even distance away from  $\vec{p}$  in opposite directions, and then computing the angle of the hypotenuse connecting them (or, essentially, the slope-angle of the road on a 2-D map) as

$$\theta_n = \arctan\left(\frac{p_{2x} - p_{1x}}{p_{2y} - p_{1y}}\right) + \frac{\pi}{2}n \quad (1)$$

where  $n = 0, 1, 2, 3$  and  $x$  and  $y$  refer to longitude and latitude respectively (in a projected/length-conserving coordinate reference system for the local area).

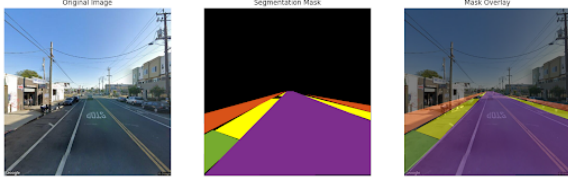
Using this method, 200 GPS points were sampled randomly throughout the testing region to develop a training dataset. Points on intersections were not sampled due to intersections having proportionately less on-street parking than internal road sections (see appendix A.3). We opted to leave the final uniform distribution to applying the model later, so in sampling randomly, we hope to introduce some additional randomness and redundancy that would be more present in realistic scenarios due to an uneven distribution of SVI data for cities. However, to limit the random oversampling of longer roads, we weighed the selection probability for each road individually by its length, so sections of smaller roads had higher likelihoods of selection over longer roads. As such, the resulting distribution of sampled points for training is plotting in figure 3.



**Figure 3:** The distribution of 200 GPS points that were sampled to produce the model training and testing dataset of 800 SVIs.

As stated above, these 200 GPS points correspond to 800 SVIs, which were each chosen to have a resolution of 640 by 640 pixels. Each image is then manually labeled into five classes: roadside parking zones, roadside non-parking (restricted) zones, the roadway, driveways, and the sidewalk. Non-parking zones include roadside sections that cannot support legal, long-term parking, such as fire lanes, sections in

front of driveways, street crossing sections, etc. Driveways include any drivable areas that are not roadways, such as parking lots. An example of this can be seen in figure 4.



**Figure 4:** An example labeled SVI image used for the training/testing dataset, where orange denotes sidewalks, yellow denotes parking, green denotes restricted zones, and purple is the roadway.

### 3.2 Segmentation Model Development

The dataset was then split to allocate 90% and 10% of the images for training and testing respectively, and using the Albumentations library, the transformations described in table 1 were applied probabilistically to the dataset [23]. These transformations were chosen to represent the range of pictures that could be taken of the same location under varied conditions of weather, time-of-day, dimensions, camera orientation, and likewise which would alter the color profile and image orientation.

| Transformation      | Parameters                                                   | $p$ |
|---------------------|--------------------------------------------------------------|-----|
| Horizontal Flip     | N/A                                                          | 50% |
| Affine Transform    | Scale $\pm 5\%$ , Translate $\pm 5\%$ , Rotate $\pm 7^\circ$ | 50% |
| Random Crop         | 0-10% (Aspect Ratio 0.9-1.1)                                 | 25% |
| Brightness/Contrast | $\pm 30\%$ (both)                                            | 50% |
| HSV Shift           | Hue $\pm 8$ , Saturation $\pm 15$ , Value $\pm 15$           | 40% |
| Gaussian Blur       | Kernel Size of 3-5                                           | 15% |
| Gaussian Noise      | N/A                                                          | 20% |

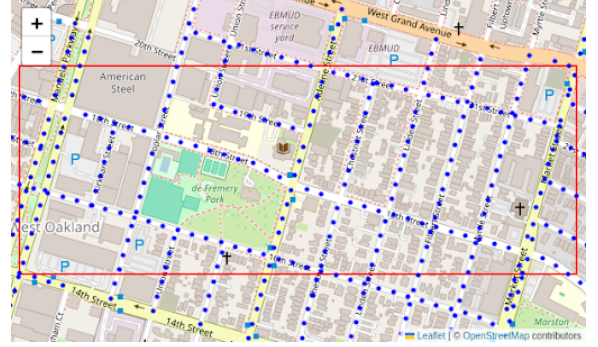
**Table 1:** Albumentations transformations used during training.  $p$  is the probability of a transformation being applied.

Then, using the HuggingFace library, the NVIDIA MIT-B2 SegFormer instance (where only the encoder is pre-trained on the ImageNet-1K database) was loaded for fine-tuning [24, 25]. This model architecture was chosen due to its simplicity and efficiency, as its compact architecture and comparably low number of parameters allow for the model to be quickly trained and easily stored [26]. The model was then trained on Google Colaboratory using an L4 GPU and 50 gigabytes of memory for 60 epochs using a batch size of 8, an initial learning rate of  $6 \times 10^{-5}$ , weight decay of 1%, and a warmup ratio of 10%, then finally evaluated on the testing dataset

to determine the accuracy and IoU (intersection over union) across the two classes of interest: parking and non-parking zones.

### 3.3 Map Generation

The road geometry map from figure 2 was then discretized by a spacing of 25 meters into uniformly-spaced sampling points (excluding intersections), as seen in figure 5.



**Figure 5:** The road geometry map discretized into uniformly-spaced sampling points with a spacing of 25 meters, excluding intersections.

For each point, images are taken from each of the four principal directions and segmentations are produced using the previously trained model. For each image, the labels for parking and non-parking (restricted) zones are then extracted, polygonized, and simplified using `findContours` and `approxPolyDP` from OpenCV to produce a polygon in the form of a short list of image coordinates bounding the shape (see appendix A.4) [27].

Then, using the CameraTransform library, the associated heading direction and GPS position for every image alongside the intrinsic/constant camera parameters of FOV (field-of-view), camera height, and tilt can be used to build a pinhole camera model to homographically transform the polygon vertices from image space and into GPS space [28]. The FOV and Tilt are well-defined and kept constant for each image at 60 and 0 degrees respectively (aligned with the horizon). Unfortunately, the exact height of the camera used to produce the SVIs obtained throughout this project isn't published by Google, so Google Street View cameras are commonly estimated to have a height of roughly 8.6 feet or 2.5 meters (and in general, estimates place the camera between 2-3 meters tall) [29, 30]. This inaccuracy will be discussed in greater detail in section 5.3.

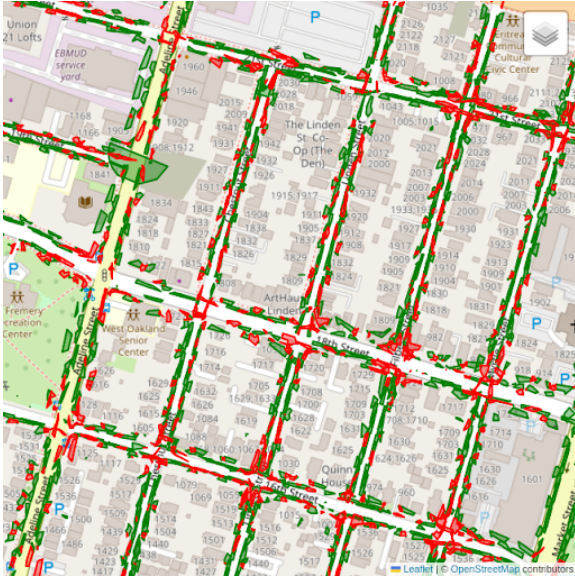
For each image, this camera model is reconstructed and all of the labels for the parking and non-parking zones are transformed out of the image space and into GPS space using the `gpsFromImage` function of CameraTransform [28] (see appendix A.5). Due to the transform becoming more

inaccurate at farther distances (i.e., closer to the horizon), we then filter out the resulting GPS polygon to exclude any area produced outside of a 30 meter buffer. This full process can be seen in figure 6.



**Figure 6:** Each stage of the homographic transform. Green and red polygons are parking and non-parking (restricted) zones respectively. Left: The initial photo. Middle: The photo after being segmented for on-street parking and restricted area. Right: The segmentation polygons after being transformed into GPS space.

Then, this process is applied across all directions for all of the discretized points in the application dataset to produce a map of all immediate labels using the aforementioned procedure (see appendix A.6), and this process produces a map of raw GPS polygons, with many overlapping, which can be seen in figure 7.

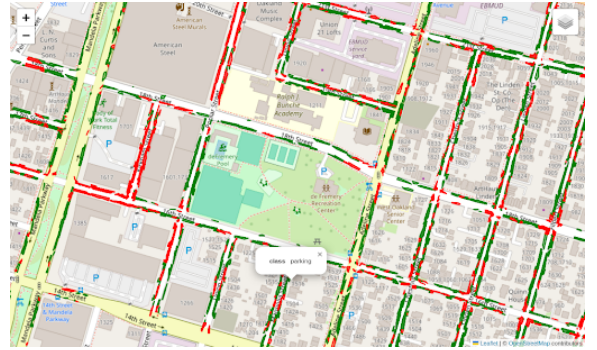


**Figure 7:** The raw GPS segmentation map showing overlapping polygons representing parking locations, generated by applying the segmentation process described in figure 6 across the full application area prior to post-processing.

Then, this dataset undergoes a number of post-processing transforms to produce the final dataset. First, small polygons under 5 square meters in area are removed due to being unable to support any realistic development, as for reference, compact cars are generally 5.4 square meters or larger in area [31].

Then, any overlapping polygons of the same class are combined, and the intersection between the non-parking and parking layers is removed from the parking layer, producing a conservative estimate for where the on-street parking is (as regardless of the number of predictions for on-street parking, the presence of a non-parking label will completely override the other predictions).

Lastly, by assuming that all of the roads are the same width and perfectly straight, we assume that parking cannot exist within a 3 meter distance from the center of the road and farther than an 8 meter distance from the center of the road, and this filters out any predictions that lie directly on roadways, sidewalks, or buildings. This produces the final map, which can be seen in figure 8.



**Figure 8:** The final map produced from the entire process of segmentation and post-processing. Darker red/green polygons indicates multiple overlaps, meaning that the model is more confident in its prediction for that region.

## 4 Results

### 4.1 Segmentation Model

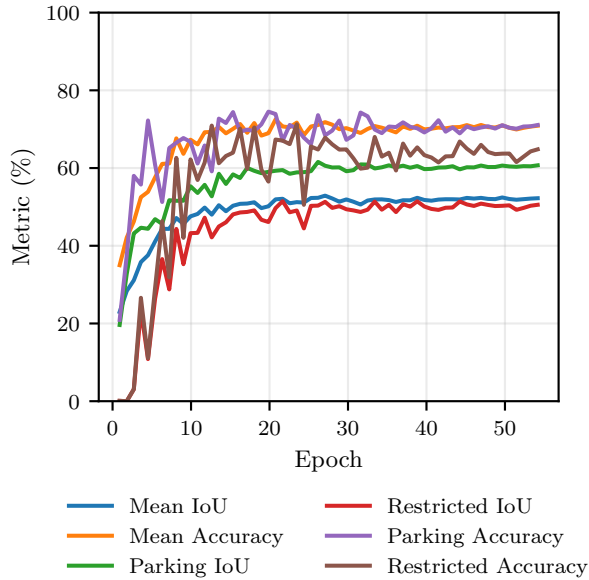
The results of the segmentation model training are tabulated in table 2. The model performs at 61% and 58% IoU for detecting the parking and non-parking regions respectively at a mean IoU of 52% and an accuracy of detecting parking of 72%. This indicates that on average, the model is able to identify valid parking and non-parking space with moderate overlap with the ground truth while failing to capture the full area and shape of most parking and non-parking zones. This means that the final map produced from the segmentation model will likely be a correct general guideline for the locations of on-street parking zones, but the areas and shapes of the produced polygons will likely be very error-prone.

Furthermore, the metrics shown in table 2 and the hyperparameters discussed in section 3.2 which produced these metrics are validated by figure 9, which shows that after roughly 20 to 30 epochs, the model converged to the accuracy

| Class           | IoU (%)      | Accuracy (%) |
|-----------------|--------------|--------------|
| Sidewalk        | 79.53        | 86.09        |
| Parking         | 61.06        | 71.65        |
| Roadway         | 90.39        | 96.22        |
| Restricted Zone | 58.30        | 65.89        |
| Driveway        | 32.34        | 36.34        |
| <b>Mean</b>     | <b>52.38</b> | <b>71.25</b> |

**Table 2:** Final validation performance of the semantic segmentation model, showing accuracy and Intersection over Union (IoU) metrics for parking and non-parking regions.

and precision possible given the size of the present training dataset for each of the metrics of interest (such as parking, non-parking, and mean IoU and accuracy).



**Figure 9:** Training metrics as a function of epoch for the SegFormer B-2 model, including the mean IoU (mIoU), mean accuracy, and IoU and accuracy for the parking and non-parking classes.

## 5 Discussion

In evaluating the effectiveness of the full data analysis process we are proposing, we must remember that the goal of this process is to be a reliable, accurate, cheap, and scalable process for developing geographic maps of on-street parking land. This means that a holistic approach must be taken to evaluating our prior work, considering the realistic requirements and tolerances in accuracy, precision, time, cost, area, and likewise that any public or private stakeholders that may

wish to employ this process would have. As such, we choose to focus on three critical areas for analysis: the performance of the segmentation model, the effectiveness of the process as applied to the test case, and the potential of this process for realistic application.

### 5.1 Model Performance

Although the produced model performance displayed in section 4.1 is an objectively poor score for most segmentation models, it is an acceptable score for this task for two main reasons: perspective redundancy and a high error tolerance in the final map.

First, each section of the roadway is actually segmented multiple times due to being observed at multiple perspectives at varying distances. Second, the final map does not need to be any more precise than a rough approximation of the locations of on-street parking to produce a final actionable map through post-processing techniques, as these allow the shapes produced following the segmentation and homographic transform to be wholly inaccurate (as seen in figure 7). Furthermore, for the final map to be at all useful for developers, it does not necessarily need to be fully precise in the final area and shape of each segmentation or to accurately determine all available on-street parking spaces within the application region, as the goal of the process is to simply provide an accurate guide for locating on-street parking space within a region.

### 5.2 Strengths

The greatest strength of this method, in all, lies in its practicality. This practicality is developed through multiple factors, such as its extensibility, efficiency, and scalability. As stated in section 1, the method of using semantic segmentation on street view imagery has already been proven effective for extracting geographic information [3], and it is a particularly efficient and scalable method for extracting information when compared with alternative methods of data collection [5].

Furthermore, our investigation has proven to be particularly efficient due to the lack of specificity required in solving the problem of mapping out on-street parking spaces. As stated above in section 5.1, our lack of necessity in precision allowed for a focus on speed. First, our use of principal directions and spacing along roads leads to a lot of overlap in the images as the same geographic location will be viewed from numerous angles, allowing for any random inaccuracy in one segmentation to be verified against another segmentation produced in a different angle. Second, we used a mere 800 images and standard, low-cost methods for developing and training the model, such as choosing B-2 SegFormer (which is neither a light-weight nor heavy-weight model at

27.5 million parameters) and training the model remotely using Google Colaboratory. As such, the entire process is relatively simple, and as with any segmentation project, the primary bottle-neck throughout its development was manually labeling segmentation masks to develop the training dataset, and otherwise, the rest of the process could easily be completed in a few hours at most.

Additionally, this process can be easily tweaked or extended for greater speed or greater accuracy. A lightweight segmentation model like SegFormer B-0, at only 3.7 million parameters, could likely achieve a comparable accuracy, and as shown in figure 9, the model could use as few as twenty epochs to converge [26]. Conversely, SegFormer B-5, the present state-of-the-art with 84.4 million parameters, could be used for greater accuracy, and this may constitute the use of more epochs. Similarly, we chose a dataset containing only 800 images somewhat arbitrarily and could have been increased as needed.

Lastly, the greatest advantage is the low financial cost to implement this method. Costs for this particular project were essentially nil, requiring only a subscription to Colaboratory due to the number of Street View API calls this project made being less than the maximum given for free as we needed a relatively small number of images. However, larger, more complex mapping efforts may choose to use dedicated computing clusters and thousands to millions of images to precisely map large areas, and while this would certainly increase the cost, this would likely not be considerably expensive. For example, obtaining 5 million or more images from the Street View API would likely cost between 2 and 3 thousand dollars [32]. More importantly, through this method, we skip the costs of traditional surveying methods: transportation, measuring materials and time. The use of existing infrastructure makes the mapping process cheaper and faster than existing surveying methods. Furthermore, in environments that lack Street View data provided by Google, SVI datasets can be obtained manually through mapping the streets by leveraging cameras in everyday cars without needing highly specialized equipment, and this is especially effective through crowd sourcing data [5].

### 5.3 Limitations and Potential Improvements

Unfortunately, this method carries significant limitations that are key to understand before utilizing it. As this process is highly sequential, errors produced at each step are propagated throughout the process, leading to further errors downstream.

For example, the process of determining the principal directions relies on the assumption that the road is perfectly straight. This is a generally true assumption given our choice of application region, but in cases of curved roads, this assumption fails. In this case, however, the result is that no

images directly facing down the road would be taken (which would be harder to segment due to potentially viewing parking at off-angles), but this likely will not necessarily introduce significant error as supplementing in additional data at off-angles would result in these edge-cases being sufficiently accounted for by the model.

Additionally, the homographic transform relies on the assumption that the local area captured within a street view image is perfectly flat and very close to the location of the camera, as this allows for two-dimensional image space to be mapped to two-dimensional cartesian space which is then mapped to two-dimensional, spherical GPS space. Close blockages in front of the camera, far distances viewed on the camera, and non-negligible elevation changes (like steep slopes) are not accurately accounted for under this present model (see appendix A.5). However, this assumption is reasonable as many images can be taken to ensure that each segmentation only covers a short distance away from the camera, and given the width of common urban streets, the closest on-street parking spaces are normally going to be considerably close (within ten meters) of the camera.

Furthermore, as mentioned in section 3.3, the exact height of the camera used for each image is not known, and this leads to greater inaccuracy in the transform. However, given the approximation range of the height between 2 to 3 meters, the height inaccuracy is negligible for segmented regions physically close to the camera, and as stated before, we are already ignoring physically far regions.

Lastly, the entirety of the method, as applied in this report, is not intended to be wholly accurate due to the implicit human inaccuracy in determining where on-street parking should lie from street view images alone. For example, on-street parking can often be conditional, such as with time limits or public-access hours, and our model did not take these edge-cases into account whatsoever. Consideration of additional edge-cases should naturally require the use of additional segmentation classes in the model, and similarly, a far larger dataset than the one we have employed presently. To curb the heavy requirement for training data, one potential suggestion for a future attempt is to leverage a maximally-precise (yet computationally heavy) model such as SegFormer B-5 or InternImage (i.e., standard high-accuracy models in fields like medical segmentation) to extend a smaller foundational dataset before training a final dataset with a lighter model like SegFormer B-0.

## 6 Conclusion

In summary, we’ve developed a systematic approach to developing high-accuracy geographic maps of on-street parking land using an example scenario in Oakland posed by our com-

munity partner, Neighborhood. This process includes collecting SVIs, training a semantic segmentation model, producing image boundaries for parking and non-parking (restricted) zones along the sides of roadways, transforming these boundaries from image space into GPS space, and then performing various post-processing operations to the resultant dataset of GPS polygons to produce a final, polished map of GPS polygons bounding both parking and non-parking zones along the sides of roadways. Lastly, we analyzed this process and its results and discussed its strengths and weaknesses, noting its overall efficiency and usefulness as a quick approximation method for mapping out on-street parking using limited resources and time despite its shortcomings in accuracy and completeness while considering various improvements that could be made in various stages of the process for better accuracy and efficiency.

## 7 Acknowledgements

We would like to extend our greatest thanks to Adam Garrett-Clark, Sonja Deng, and Keilan Pieper for their advice and input on the project. Adam Garrett-Clark, our liaison with Neighborhood, was instrumental in formulating this project and overseeing its development, Sonja Deng provided useful advice in developing and applying the segmentation model, and Keilan Pieper provided the suggestion of utilizing a “heavy” model for dataset development before training a “light” model as described in section 5.3. We would also like to thank Richard Gerum, the author of CameraTransform, for his advice on how the homographic transform is produced computationally (as described in section A.5).

Furthermore, generative artificial intelligence (Anthropic’s Sonnet 5 Claude, OpenAI’s ChatGPT-5.5 and Google’s Gemini 3.3) was used heavily for code generation throughout the development process, formatting figures and tables, and typesetting this document from an initial report in Google Docs to L<sup>A</sup>T<sub>E</sub>X. Generative artificial intelligence was not used in developing the content of this report whatsoever, including the abstract, section text, and captions, but artificial intelligence-based proofreading tools like CheckMyManuscript and Grammarly were used for revising this text.

## 8 Code and Data Availability

All of the code and data used in this report are made publicly available through the GitHub repository [KalyanLabUBC/parklet-vision](https://github.com/KalyanLabUBC/parklet-vision). This includes all of the training and testing data, data analysis scripts (such as the one used to produce figure 9), and more.

The most important sections of the code are structured into three easy-to-follow Jupyter notebooks for clarity,

1. `1_data_collection.ipynb`,
2. `2_fine_tuning.ipynb`, and
3. `3_crawler_classifier.ipynb`

which correspond with sections 3.1, 3.2, and 3.3 respectively.

## References

- [1] Fang Zhao et al. “Smart city research: A holistic and state-of-the-art literature review”. In: *Cities* 119 (2021), p. 103406. ISSN: 0264-2751. DOI: <https://doi.org/10.1016/j.cities.2021.103406>. URL: <https://www.sciencedirect.com/science/article/pii/S026427512100305X>.
- [2] Jose Sanchez Gracias et al. “Smart Cities—A Structured Literature Review”. In: *Smart Cities* 6.4 (2023), pp. 1719–1743. ISSN: 2624-6511. DOI: [10.3390/smartcities6040080](https://www.mdpi.com/2624-6511/6/4/80). URL: <https://www.mdpi.com/2624-6511/6/4/80>.
- [3] Chaimae Aajal et al. “Autonomous On-Street Parking Slot Detection: A Comprehensive Systematic Review of Vision-Based Approaches”. In: *IEEE Access* 13 (2025), pp. 183734–183754. DOI: [10.1109/ACCESS.2025.3623566](https://doi.org/10.1109/ACCESS.2025.3623566).
- [4] Cristian Roman et al. “Detecting On-Street Parking Spaces in Smart Cities: Performance Evaluation of Fixed and Mobile Sensing Systems”. In: *IEEE Transactions on Intelligent Transportation Systems* 19.7 (2018), pp. 2234–2245. DOI: [10.1109/TITS.2018.2804169](https://doi.org/10.1109/TITS.2018.2804169).
- [5] Shatha Alahmadi et al. “Towards Scalable and Privacy-Enhanced On-Street Parking Management: A Roadmap for Future Inquiry”. In: *Electronics* 12.19 (2023). ISSN: 2079-9292. DOI: [10.3390/electronics12194160](https://doi.org/10.3390/electronics12194160). URL: <https://www.mdpi.com/2079-9292/12/19/4160>.
- [6] Jhonattan J. Barriga et al. “Smart Parking: A Literature Review from the Technological Perspective”. In: *Applied Sciences* 9.21 (2019). ISSN: 2076-3417. DOI: [10.3390/app9214569](https://doi.org/10.3390/app9214569). URL: <https://www.mdpi.com/2076-3417/9/21/4569>.
- [7] *Parklet Program*. URL: <https://www.oaklandca.gov/Public-Safety-Streets/Parking-Vehicles/Parklet-Program>.

- [8] Jasmine Ford and Lilia Gomez-Lanier. “Are Tiny Homes Here to Stay? A Review of Literature on the Tiny House Movement”. In: *Family and Consumer Sciences Research Journal* 45.4 (2017), pp. 394–405. DOI: <https://doi.org/10.1111/fcsr.12205>. eprint: <https://onlinelibrary.wiley.com/doi/pdf/10.1111/fcsr.12205>. URL: <https://onlinelibrary.wiley.com/doi/abs/10.1111/fcsr.12205>.
- [9] *Neighborhood.org*. URL: <https://www.neighborhood.org>.
- [10] Rui Li, Alysha Helmrich, and Mikhail Chester. “Parking by the bay: The supply and implications of parking infrastructure in the San Francisco Bay Area”. In: *Journal of Transport and Land Use* 15.1 (Aug. 2022), pp. 527–545. DOI: 10.5198/jtlu.2022.2123. URL: <https://www.jtlu.org/index.php/jtlu/article/view/2123>.
- [11] Filip Biljecki and Koichi Ito. “Street view imagery in urban analytics and GIS: A review”. In: *Landscape and Urban Planning* 215 (2021), p. 104217. ISSN: 0169-2046. DOI: <https://doi.org/10.1016/j.landurbplan.2021.104217>. URL: <https://www.sciencedirect.com/science/article/pii/S0169204621001808>.
- [12] Konstantinos Gkolias and Eleni I. Vlahogianni. “Convolutional Neural Networks for On-Street Parking Space Detection in Urban Networks”. In: *IEEE Transactions on Intelligent Transportation Systems* 20.12 (2019), pp. 4318–4327. DOI: 10.1109/TITS.2018.2882439.
- [13] Sezer Gören et al. “On-Street Parking Spot Detection for Smart Cities”. In: *2019 IEEE International Smart Cities Conference (ISC2)*. 2019, pp. 292–295. DOI: 10.1109/ISC246665.2019.9071760.
- [14] Yifan Pu et al. “City-scale roadside electric vehicle parking and charging capacity: A deep learning augmented street-view-image data mining and analytic framework”. In: *Applied Energy* 389 (2025), p. 125795. ISSN: 0306-2619. DOI: <https://doi.org/10.1016/j.apenergy.2025.125795>. URL: <https://www.sciencedirect.com/science/article/pii/S0306261925005252>.
- [15] Krisna R. Sijabat et al. “Integrating Remote Sensing and Street View Imagery with Deep Learning for Urban Slum Mapping: A Case Study from Bandung City”. In: *Applied Sciences* 15.14 (2025), p. 8044. URL: <https://www.proquest.com/scholarly-journals/integrating-remote-sensing-street-view-imagery/docview/3233064320/se-2>.
- [16] Romi Fadillah Rahmat et al. “Extreme Learning Machine for Parking Slot Recognition in Smart Parking System”. In: *2024 8th International Conference on Electrical, Telecommunication and Computer Engineering (ELTICOM)*. 2024, pp. 113–118. DOI: 10.1109/ELTICOM64085.2024.10865423.
- [17] Bradley Koskovich, Michael Starek, and Scott A. King. “VirtualLot—A Case Study on Combining UAS Imagery and Terrestrial Video with Photogrammetry and Deep Learning to Track Vehicle Movement in Parking Lots”. In: *Remote Sensing* 14.21 (2022), p. 5451. URL: <https://www.proquest.com/scholarly-journals/virtualot-case-study-on-combining-uas-imagery/docview/2771655456/se-2>.
- [18] J.-Emeterio Navarro-B., Martin Gebert, and Ralf Bielig. “On automatic extraction of on-street parking spaces using park-out events data”. In: *2021 IEEE International Conference on Omni-Layer Intelligent Systems (COINS)*. 2021, pp. 1–7. DOI: 10.1109/COINS51742.2021.9524119.
- [19] C. Henry et al. “CITYWIDE ESTIMATION OF PARKING SPACE USING AERIAL IMAGERY AND OSM DATA FUSION WITH DEEP LEARNING AND FINE-GRAINED ANNOTATION”. English. In: *The International Archives of Photogrammetry, Remote Sensing and Spatial Information Sciences XLIII-B2-2021* (2021), pp. 479–485. URL: <https://www.proquest.com/conference-papers-proceedings/citywide-estimation-parking-space-using-aerial/docview/2585322701/se-2>.
- [20] Jens Hellekes et al. “Parking space inventory from above: Detection on aerial images and estimation for unobserved regions”. In: *IET Intelligent Transport Systems* 17.5 (2023), pp. 1009–1021. DOI: <https://doi.org/10.1049/itr2.12322>. eprint: <https://ietresearch.onlinelibrary.wiley.com/doi/pdf/10.1049/itr2.12322>. URL: <https://ietresearch.onlinelibrary.wiley.com/doi/abs/10.1049/itr2.12322>.
- [21] Howard Butler et al. *The GeoJSON Format*. RFC 7946. Internet Engineering Task Force, Aug. 2016. DOI: 10.17487/RFC7946. URL: <https://www.rfc-editor.org/rfc/rfc7946>.

- [22] Kelsey Jordahl et al. *geopandas/geopandas: v1.1.4*. Version v0.8.1. July 2020. DOI: [10.5281/zenodo.3946761](https://doi.org/10.5281/zenodo.3946761). URL: <https://doi.org/10.5281/zenodo.3946761>.
- [23] Alexander Buslaev et al. “Albumentations: Fast and Flexible Image Augmentations”. In: *Information* 11.2 (2020). ISSN: 2078-2489. DOI: [10.3390/info11020125](https://doi.org/10.3390/info11020125). URL: <https://www.mdpi.com/2078-2489/11/2/125>.
- [24] Thomas Wolf et al. *HuggingFace’s Transformers: State-of-the-art Natural Language Processing*. 2020. arXiv: [1910.03771](https://arxiv.org/abs/1910.03771) [cs.CL]. URL: <https://arxiv.org/abs/1910.03771>.
- [25] Jia Deng et al. “ImageNet: A large-scale hierarchical image database”. In: *2009 IEEE Conference on Computer Vision and Pattern Recognition*. 2009, pp. 248–255. DOI: [10.1109/CVPR.2009.5206848](https://doi.org/10.1109/CVPR.2009.5206848).
- [26] Enze Xie et al. “SegFormer: simple and efficient design for semantic segmentation with transformers”. In: *NIPS ’21*. Red Hook, NY, USA: Curran Associates Inc., 2021. ISBN: 9781713845393.
- [27] Ivan Culjak et al. “A brief introduction to OpenCV”. In: *2012 Proceedings of the 35th International Convention MIPRO*. 2012, pp. 1725–1730.
- [28] Richard C. Gerum et al. “CameraTransform: A Python package for perspective corrections and image mapping”. In: *SoftwareX* 10 (2019), p. 100333. ISSN: 2352-7110. DOI: <https://doi.org/10.1016/j.softx.2019.100333>. URL: <https://www.sciencedirect.com/science/article/pii/S2352711019302018>.
- [29] Michael Zhang. *A glimpse of Google’s fleet of camera-equipped street view cars*. Nov. 2025. URL: <https://petapixel.com/2012/10/15/a-glimpse-of-googles-fleet-of-camera-equipped-street-view-cars/>.
- [30] Google Wikia Contributors. URL: [https://google.fandom.com/wiki/Google\\_Street\\_View](https://google.fandom.com/wiki/Google_Street_View).
- [31] David Ibiknle. *Average Car Sizes: Length, Width, and Height*. May 2024. URL: <https://www.neighbor.com/storage-blog/average-car-sizes-dimensions/>.
- [32] Google. *Google Maps Platform Core Services Pricing List*. Google. 2026. URL: <https://developers.google.com/maps/billing-and-pricing/pricing#map-loads-pricing> (visited on 08/25/2026).
- [33] Richard Gerum. *CameraTransform Documentation*. Accessed: 2026-09-04. 2024. URL: <https://cameratransform.readthedocs.io>.
- [34] Iñaki Rabanillo. *Projective Geometry: Building the Homography Matrix from Scratch*. Accessed: 2025-11-26. Apr. 2023. URL: [https://inakiraba91.github.io/posts/projective\\_geometry/building\\_homography\\_matrix/](https://inakiraba91.github.io/posts/projective_geometry/building_homography_matrix/).
- [35] R. I. Hartley and A. Zisserman. *Multiple View Geometry in Computer Vision*. Second. Cambridge University Press, ISBN: 0521540518, 2004.
- [36] Google. ‘Street View Ready Pro’ Specifications. Google for Developers. 2024. URL: <https://developers.google.com/streetview/ready/specs> (visited on 08/12/2026).

## A Code Snippets and Pseudocode

### A.1 Overpass Turbo Query for Road Geometry of Oakland

The following script was used to query OpenStreetMap for all of the road-lines throughout Oakland.

```
[out:json][timeout:25];

area["name"="Oakland"]
["boundary"="administrative"]->.searchArea;

(
  way["highway"~"motorway|trunk|primary|secondary|tertiary|residential"]
  (area.searchArea);
);

out body;
>;
out skel qt;
```

### A.2 Principal Directions Calculation

Algorithm 1 is, in essence, a computational form of equation 1. It computes the principal directions for a given road point by computing the tangent angle associated with the change in latitude and longitude produced by going a uniform distance  $\delta$  in each direction down the road (thus why this only works if the road is straight).

As LineStrings in GeoJSON format as interfaced through GeoPandas are defined by a root point interpolated by a distance, the evaluation point  $\vec{p}$  to compute the principal direc-

tions for is defined based on its interpolation distance  $d$  (i.e., how far "down the road"  $\vec{p}$  is) [21, 22]. In other words,  $\vec{p}$  could be defined as  $\text{INTERPOLATE}(R, d)$  in the pseudocode below.

---

**Algorithm 1** Calculate Principal Directions (PRINCIPALDIR)
 

---

**Require:**  $R$  is the road-line (a GeoJson LineString).

**Require:**  $d$  is the interpolation distance for the evaluation point  $\vec{p}$ .

**Require:**  $\delta$  is the buffer distance.

**Ensure:**  $\delta \ll \text{LENGTH}(R)$

$\vec{p}_1 = \text{INTERPOLATE}(R, d + \delta)$   
 $\vec{p}_2 = \text{INTERPOLATE}(R, d - \delta)$

$\Delta x = p_{2x} - p_{1x}$   
 $\Delta y = p_{2y} - p_{1y}$

$\theta = \arctan(\Delta x / \Delta y)$

**return**  $\{\theta, \theta + \pi/2, \theta + \pi, \theta + 3\pi/2\} \pmod{2\pi}$

---

### A.3 Random Point Sampling Function

Algorithm 2 is used to randomly sample GPS points along roadways (GeoJSON LineStrings) within a greater dataset of roads (i.e., a GeoPandas table) using the interpolation function  $\text{INTERPOLATE}$  defined above in appendix A.2.

Assuming that the dataset  $D$  is already clipped to contain only the roads we would wish to sample, we first obtain a random road-line  $R$  within the dataset, pick a random point  $\vec{p}$  along the road by interpolating by a distance  $d$  within  $[\delta, L - \delta]$  where  $L$  is the length of the road and  $\delta = L/\alpha$  is a buffer length to avoid sampling the ends of the road (where intersections are more likely to occur) for some scalar  $\alpha \gg 1$ , and last, we ensure the point does not lie on an intersection. With these done, the point has been successfully randomly sampled, so we then compute the principal directions using  $\text{PRINCIPALDIR}(R, d, \delta)$  defined above in appendix A.2.

Points are determined to be on intersections based on the number of intersecting roads at that point. In essence, if we were to divide up all roads  $R \in D$  that contain  $\vec{p}$  at  $\vec{p}$  then count the number of new roads which now end at  $\vec{p}$  as  $N$ , a regular road would always have either  $N = 1$  (if  $\vec{p}$  is an endpoint of  $R$ ) or  $N = 2$  (meaning that  $\vec{p}$  lies internally in  $R$ ), and an intersection would always have  $N \geq 3$ . Therefore, to curb intersections, we simply calculate  $N$  for  $\vec{p}$  on  $D$ , and check if  $N \geq 3$ .

---

**Algorithm 2** Random Sample Road Point
 

---

**Require:**  $D$  is the dataset of road-lines (a list of LineStrings).

**Require:**  $\alpha$  is a scalar used to compute a buffer for the road.

**Ensure:**  $D$  is already clipped to the target region.

**Ensure:**  $\alpha \gg 1$  ( $\alpha := 32$  in the actual implementation)

$R = \text{SAMPLEROAD}(D)$   
 $L = \text{LENGTH}(R)$   
 $\delta = L/\alpha$   
 $d = \text{SAMPLE}([\delta, L - \delta])$   
 $\vec{p} = \text{INTERPOLATE}(R, d)$

**if**  $\vec{p}$  lies on an intersection **then**  
     Repeat from sampling  $d$   
**end if**

**return**  $\vec{p}, \text{PRINCIPALDIR}(R, d, \delta)$

---

### A.4 Bounding Boxes Generator

Algorithm 3 is used to produce the image-space simplified polygons demarcating the parking and non-parking classes from raw segmentation masks (which are raw mappings between each pixel coordinate  $\vec{x}$  to a class index  $c$ ). It does so using multiple OpenCV functions, namely

1.  $\text{FINDCONTOURS}(M)$ , which produces a list of image coordinates demarcating each polygon from the pixel mask  $M$ ,
2.  $\text{ARCLENGTH}(P)$ , which computes the perimeter of a polygon  $P$ , and
3.  $\text{APPROXPOLYDP}(P, \epsilon)$ , which approximates  $P$  within a precision of  $\epsilon$  to reduce the number of points required to specify  $P$  [28].

In addition, the function to run a prediction on an image  $I$  to produce the raw segmentation mask  $M$  using the pretrained model  $f$  is defined as  $\text{SEGMENT}(I, f)$ .

### A.5 Extracting Class Boundaries in GPS (Homographic Transform)

As stated in section 3.3, the homographic transform is used to project the image-space polygons produced using the boundary generator described in appendix A.4 into GPS space (as seen in figure 6), and it is performed by building a pinhole camera model using the  $\text{CameraTransform}$  that approximates the physical camera that took the SVI [28]. To produce this transformation, two smaller transformations need to be produced.

---

**Algorithm 3** Generate Bounding Boxes (EXTRACT-BOUNDARIES)
 

---

**Require:**  $I$  is the SVI we want to produce the labels for.

**Require:**  $f$  is the pretrained segmentation model.

---

$M = \text{SEGMENT}(I, f)$

$A = \text{FINDCONTOURS}(M)$

$B$  is an empty list.

**for all**  $P \in A$  **do**

$\varepsilon = \text{ARCLENGTH}(P)/100$

$P' = \text{APPROXPOLYDP}(P, \varepsilon)$

**if**  $P'$  has  $\geq 2$  vertices **then**

$B \leftarrow P'$

**end if**

**end for**

**return**  $B$  (the list of simplified polygons).

---

For a raw SVI  $I$ , points that lie within the image can be said to lie in the “image plane” or “image space”  $\mathcal{V}$ , and these points then need to be transformed into three-dimensional “unit space”  $\mathcal{W}$ , which is a cartesian space that encodes the exact positions of each of the points in the image relative to an origin point (generally the camera). Then, we obtain points in spherical GPS space  $\mathcal{U}$  from performing a cartesian-to-spherical transformation between unit space to GPS space. The first transformation  $\mathcal{V} \rightarrow \mathcal{W}$  is defined using the first homographic transformation matrix  $\mathbf{H}_1^{-1}$  and the second transformation  $\mathcal{W} \rightarrow \mathcal{U}$  is defined using the second homographic transformation matrix  $\mathbf{H}_2^{-1}$ , and the overall homographic transformation describing the translation from image-space to GPS-space is  $\mathbf{H} : \mathcal{V} \rightarrow \mathcal{U}$ .

Additionally, to be clear, the following linear algebra formulation of the homographic transform is not exactly how it is performed in the CameraTransform library: rather, it is included here to describe a mathematical rationale for how the transform is performed in standard computer vision literature. For example, as described by Richard Gerum in personal correspondence, the CameraTransform library undergoes this transformation by casting rays, or a “line of sight that all lands in [a given] camera pixel as a 3D line definition”. For a more exact description on this transform as performed in the library, readers are directed to the CameraTransform documentation [33].

First, the intrinsic matrix describing the non-skewed and undistorted rectilinear projection (the standard pinhole cam-

era model) is

$$\mathbf{C}_{intr} = \begin{pmatrix} f_x & 0 & W/2 \\ 0 & f_y & H/2 \\ 0 & 0 & 1 \end{pmatrix} \quad (2)$$

where  $f_x, f_y$  are the focal lengths of the camera and  $W$  and  $H$  are the width and height of the image respectively [34, 28]. In detail, the focal lengths are computed from the FOV angles  $\alpha_x$  and  $\alpha_y$  of the camera as

$$f_x = \frac{W/2}{\tan(\alpha_x/2)} \quad (3)$$

$$f_y = \frac{H/2}{\tan(\alpha_y/2)}, \quad (4)$$

and this is necessary as only the FOV angles are specified for each camera and not the focal lengths [35, 36].

Then, this  $3 \times 3$  intrinsic camera matrix  $\mathbf{C}_{intr}$  is multiplied by the  $3 \times 4$  rotation and translation matrix  $[\mathbf{R} \quad \mathbf{t}]$  produced from a  $3 \times 3$ -shape 3-dimensional rotation matrix

$$\mathbf{R} = R_z(\theta_z)R_y(\theta_y)R_x(\theta_x) \quad (5)$$

produced from the three-dimensional rotation angles of roll  $\theta_x$ , tilt  $\theta_y$ , and heading  $\theta_z$  [35, 34].

Together, this produces the  $3 \times 4$  3-D to image homographic transformation matrix  $\mathbf{H}_1 : \mathcal{V} \rightarrow \mathcal{W}$  defined as

$$\mathbf{H}_1 = \mathbf{C}_{intr} [\mathbf{R} \quad \mathbf{t}] \quad (6)$$

that relates coordinates in three-dimensional unit space  $\mathcal{V}$  to two-dimensional image space  $\mathcal{W}$ , and this produces the equation

$$s\vec{\mathbf{w}} = \mathbf{H}_1\vec{\mathbf{v}} \quad (7)$$

for  $\vec{\mathbf{w}} \in \mathcal{W}$  and  $\vec{\mathbf{v}} \in \mathcal{V}$  where  $s$  is the projective scale factor [34]. As written, this equation is invertible due to  $\mathbf{H}_1$  being  $3 \times 4$ , but this matrix can be made  $3 \times 3$  where  $\mathbf{H}$  represents a homographic mapping between two two-dimensional planes, and this is what we seek to do in converting between the  $X/Y$  image plane and the  $x/y$  unit-space plane as we assume that all points lie at  $z = 0$  [34]. This allows the matrix to be invertible, and the matrix  $\mathbf{H}_1^{-1} : \mathcal{W} \rightarrow \mathcal{V}$  is used to convert between image-space coordinates and three-dimensional projections. In particular, this means that the conversion from an image-space point  $\vec{\mathbf{w}} = (X, Y)$  to the three-dimensional unit space point  $\vec{\mathbf{v}} = (x, y, 0)$  is

$$s\mathbf{H}_1^{-1} \begin{bmatrix} X \\ Y \\ 1 \end{bmatrix} = \begin{bmatrix} x \\ y \\ 1 \end{bmatrix} \quad (8)$$

[34].

The preceding description describes the first transformation from image space  $\mathcal{W}$  to three-dimensional space  $\mathcal{V}$ , and

the subsequent transformation from three-dimensional space to GPS space  $\mathcal{U}$  is considerably more complicated. As described by Richard Gerum (via personal correspondence) and the CameraTransform documentation, this transformation is produced by computing the physical distance and direction to each point then computing the necessary latitude and longitude coordinates based on the camera's GPS position [33].

For example, say we have a point  $\vec{v} \in \mathcal{V}$  and wish to compute its latitude  $\phi_1$  and longitude  $\lambda_1$ . The camera is situated at  $h\hat{z}$  with altitude  $h$  (or, more importantly, at the origin when projected to the  $x, y$ -plane), a GPS position of  $\phi_0$  latitude and  $\lambda_0$  longitude. To begin, the bearing  $\beta$  of  $\vec{v}$  (the relative heading between the line of sight of the camera and  $\vec{v}$ ) is computed as

$$\beta = \arctan(v_x/v_y) \quad (9)$$

and the  $x, y$ -planar distance  $d$  to  $\vec{v}$  is computed as

$$d = \sqrt{v_x^2 + v_y^2} \quad (10)$$

[33]. From this, the latitude  $\phi_1$  is computed as

$$\phi_1 = \arcsin \left\{ \sin \phi_0 \cos \frac{d}{R} + \cos \phi_0 \sin \frac{d}{R} \cos \beta \right\} \quad (11)$$

and the longitude  $\lambda_1$  is computed as

$$\lambda_1 = \lambda_0 + \arctan \left\{ \frac{\sin \beta \sin \frac{d}{R} \cos \phi_0}{\cos \frac{d}{R} - \sin \phi_0 \sin \phi_1} \right\} \quad (12)$$

where  $R = 6371$  km is the mean radius of the Earth [33].

## A.6 Applying the Segmentations Across the Full Area

Algorithm 4 is used to combine the procedures covered in appendices A.2, A.3, A.4, and A.5 to produce a raw GPS segmentation map (figure 7) to later post-process into the final map (figure 8). In this algorithm, the function  $\text{CAMERA}(\vec{p}, \theta)$  refers to the construction of the precise rectilinear camera model located at  $\vec{p}$  with heading  $\theta$ .

---

### Algorithm 4 Build Raw Segmentation Map

---

**Require:**  $D$  is the dataset of road-lines (a list of LineStrings).

**Require:**  $\ell = 30$  is the spacing between points.

**Require:**  $f$  is the pretrained segmentation model.

**Ensure:**  $D$  is already clipped to the target region.

$P = \{ \vec{p} \in D : \text{Each point } \vec{p} \text{ is spaced by } \ell \text{ meters.} \}$

**for all**  $\vec{p} \in P$  **do**

**if**  $\vec{p}$  is on an intersection **then**

        Remove  $\vec{p}$  from  $P$

**end if**

**end for**

$X$  is an empty list of GPS polygons.

**for all**  $\vec{p} \in P$  **do**

$R \in D$  is the road associated with  $\vec{p}$ .

$L = \text{LENGTH}(R)$

$d$  is the distance associated with  $\vec{p}$  given  $R$ .

$\delta = L/32$

**for all**  $\theta_n \in \text{PRINCIPALDIR}(R, d, \delta)$  **do**

$I$  is the SVI sampled at  $\vec{p}$  facing  $\theta_n$ .

$B = \text{EXTRACTBOUNDARIES}(I, f)$

$C = \text{CAMERA}(\vec{p}, \theta_n)$

$X \leftarrow \text{TRANSFORM}(C, B)$

**end for**

**end for**

**return**  $X$

---